

1. A university maintains the marks of students in an array. During result processing, the examination cell needs to:

a) Identify the highest mark among three toppers.

b) Search for a particular student's mark record in the list.

Develop C programs using functions and pointers to perform both tasks efficiently.

```
#include <stdio.h>
int findHighest(int *arr, int *n1)
{
    int max=*arr,i;
    for(i=1;i<*n1;i++)
    {
        if(*arr>max)
        {
            max=*arr;
        }
        arr++;
    }
    return max;
}
int searchMark(int *arr, int n, int key)
{
    for (int i = 0; i < n; i++)
    {
        if (*(arr + i) == key)
            return i;
    }
    return -1;
}
int main()
{
    int marks[100];
    int n, key, position;
    printf("Enter number of students: ");
    scanf("%d", &n);
    printf("Enter marks of %d students:\n", n);
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &marks[i]);
    }
    printf("\nHighest mark among three toppers = %d\n",findHighest(marks,&n));
```

```
printf("\nEnter the mark to search: ");
scanf("%d", &key);
position = searchMark(marks, n, key);
if (position != -1)
{
    printf("Mark %d found at position %d\n", key, position + 1);
}
else
{
    printf("Mark %d not found\n", key);
}
return 0;
}
```

OUTPUT

Enter number of students: 5

Enter marks of 5 students:

10

30

100

50

45

Highest mark among three toppers = 100

Enter the mark to search: 30

Mark 30 found at position 2

2. A) Write a program to find the maximum element in each row of a matrix using pointers.

```
#include <stdio.h>
void maximum(int (*p)[10],int *r,int *c)
{
    int i=0,j,max=0,res[20];
    for(i=0;i<*r;i++)
    {
        for(j=0;j<*c;j++)
        {
            if(*(*(p+i)+j)>max)
            {
                max=(*(*(p+i)+j));
            }
        }
        res[i]=max;
        max=0;
    }
    printf("Maximum element in each row is:\n");
    for(i=0;i<*r;i++)
    {
        printf("%d\n",res[i]);
    }
}
int main()
{
    int arr[10][10],m,n,i,j;
    printf("Enter the number of rows and coloumns\n");
    scanf("%d%d",&m,&n);
    printf("Enter the elemnts to the matrix\n");
    for(i=0;i<m;i++)
    {
        for(j=0;j<n;j++)
            scanf("%d",&arr[i][j]);
    }
    maximum(arr,&m,&n);
    return 0;
}
```

Output

Enter the number of rows and coloumns

2

3

Enter the elements to the matrix

1

2

3

4

5

6

Maximum element in each row is:

3

6

- 2. B) Define a structure TIME with hour, minute, second, and a self-referencing pointer. Dynamically create two variables, link them, and display in h:m:s format.**

```
#include <stdio.h>
#include <stdlib.h>
struct time
{
    int h,m,s;
    struct time *next;
};
void display(struct time *);
int main()
{
    struct time *t1,*t2;
    t1=malloc(sizeof(struct time));
    printf("Enter the hour,minute and second for time1\n");
    scanf("%d%d%d",&t1->h,&t1->m,&t1->s);
    t2=malloc(sizeof(struct time));
    printf("Enter the hour,minute and second for time2\n");
    scanf("%d%d%d",&t2->h,&t2->m,&t2->s);
    t1->next=t2;
    display(t1);
}
void display(struct time *t)
{
    printf("%d:%d:%d\n",t->h,t->m,t->s);
    t=t->next;
```

```
    printf("%d:%d:%d",t->h,t->m,t->s);  
}
```

Output:

Enter the hour,minute and second for time1

1

30

20

Enter the hour,minute and second for time2

2

5

60

1:30:20

2:5:60

3. Write a C program using structures and functions to manage bank account holders. The program should provide the following operations:

- a. Create multiple bank accounts.**
- b. Display account details.**
- c. Deposit money into an account.**
- d. Withdraw money from an account.**
- e. Search for an account using the account number.**
- f. Calculate the total balance of all accounts.**

```
#include <stdio.h>
struct Bank
{
    int accno;
    char name[20];
    float balance;
};
void display(struct Bank b)
{
    printf("\nAccount No: %d", b.accno);
    printf("\nName: %s", b.name);
    printf("\nBalance: %.2f\n", b.balance);
}
int main()
{
    struct Bank b[10];
    int n, i, choice, acc, found;
    float amount, total = 0;

    printf("Enter number of accounts: ");
    scanf("%d", &n);

    // Create accounts
    for(i = 0; i < n; i++)
    {
        printf("\nEnter account number: ");
        scanf("%d", &b[i].accno);

        printf("Enter name: ");
        scanf("%s", b[i].name);

        printf("Enter balance: ");
```

```
        scanf("%f", &b[i].balance);
    }

    // Menu
    do
    {
        printf("\n\n1. Display Accounts");
        printf("\n2. Deposit");
        printf("\n3. Withdraw");
        printf("\n4. Search Account");
        printf("\n5. Total Balance");
        printf("\n6. Exit");

        printf("\nEnter choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                for(i = 0; i < n; i++)
                    display(b[i]);
                break;

            case 2:
                printf("Enter account number: ");
                scanf("%d", &acc);

                for(i = 0; i < n; i++)
                {
                    if(b[i].accno == acc)
                    {
                        printf("Enter amount: ");
                        scanf("%f", &amount);
                        b[i].balance += amount;
                        printf("Deposit successful!");
                    }
                }
                break;

            case 3:
                printf("Enter account number: ");
```

```

scanf("%d", &acc);

for(i = 0; i < n; i++)
{
    if(b[i].accno == acc)
    {
        printf("Enter amount: ");
        scanf("%f", &amount);
        if(amount <= b[i].balance)
        {
            b[i].balance -= amount;
            printf("Withdrawal successful!");
        }
        else
            printf("Insufficient balance!");
    }
}
break;
case 4:
    printf("Enter account number: ");
    scanf("%d", &acc);
    found = 0;
    for(i = 0; i < n; i++)
    {
        if(b[i].accno == acc)
        {
            display(b[i]);
            found = 1;
        }
    }
    if(found == 0)
        printf("Account not found!");
    break;
case 5:
    total = 0;
    for(i = 0; i < n; i++)
        total += b[i].balance;
    printf("Total Balance = %.2f", total);
    break;
case 6:
    printf("Thank you!");

```

```
        break;

        default:
            printf("Invalid choice!");
    }

    } while(choice != 6);

    return 0;
}
```

Output:

Enter number of accounts: 2

Enter account number: 12345

Enter name: Rahul

Enter balance: 1000

Enter account number: 58692

Enter name: Rohith

Enter balance: 2000

1. Display Accounts
2. Deposit
3. Withdraw
4. Search Account
5. Total Balance
6. Exit

Enter choice: 1

Account No: 12345

Name: Rahul

Balance: 1000.00

Account No: 58692

Name: Rohith

Balance: 2000.00

1. Display Accounts
2. Deposit

3. Withdraw
4. Search Account
5. Total Balance
6. Exit

Enter choice: 2

Enter account number: 12345

Enter amount: 3000

Deposit successful!

1. Display Accounts
2. Deposit
3. Withdraw
4. Search Account
5. Total Balance
6. Exit

Enter choice: 3

Enter account number: 58692

Enter amount: 100

Withdrawal successful!

1. Display Accounts
2. Deposit
3. Withdraw
4. Search Account
5. Total Balance
6. Exit

Enter choice: 5

Total Balance = 5900.00

1. Display Accounts
2. Deposit
3. Withdraw
4. Search Account
5. Total Balance
6. Exit

Enter choice: 6

Thank you!

...Program finished with exit code 0

Press ENTER to exit console.

4. Write a C program to implement a Singly Linked List with the following operations:

- a. Insert a node at the beginning of the list.**
- b. Insert a node at a specified position.**
- c. Delete a node by its value.**
- d. Search for a given element in the list.**
- e. Display all elements of the list.**
- f. Reverse the linked list and display the reversed list.**

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node
```

```
{
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

```
typedef structnode NODE;
```

```
NODE* insertfront(NODE *, int);
```

```
NODE* searchkey(NODE *, int);
```

```
NODE* i_pos(NODE *, int,int);
```

```
NODE* deletekey(NODE *, int);
```

```
NODE* reverse(NODE *);
```

```
void display(NODE *);
```

```
void main()
```

```
{
```

```
    NODE *start=NULL;
```

```
    int num,choice,k,p;
```

```
    while(1)
```

```
    {
```

```
        printf("Enter your choice\n");
```

```
        printf("1. Insert at front\n");
```

```
        printf("2. Insert at position\n");
```

```
        printf("3. Delete a node\n");
```

```
        printf("4. Searching a key\n");
```

```
        printf("5. Reverse");
```

```
        printf("6. Display\n");
```

```
        printf("7. Stop\n");
```

```
        scanf("%d", &choice);
```

```
        switch(choice)
```

```

    {
        case 1:
            printf(" Enter data to insert\n");
            scanf("%d",&num);
            start= insertfront(start,num);
            break;
        case 2:
            printf(" Enter data to insert\n");
            scanf("%d",&num);
            scanf("%d",&p);
            start= i_pos(start,num,p);
            break;
        case 3:
            printf(" Enter the key value");
            scanf("%d",&k);
            start= deletekey(start,k);
            break;
        case 4:
            printf(" Enter key to be searched\n");
            scanf("%d",&k);
            start= searchkey(start,k);
            break;
        case 5:
            printf(" Reversed string is\n");
            start= reverse(start);
            break;
        case 6:
            display(start);
            break;
        case 7:
            exit(0);
    }
}

NODE* insertfront(NODE *start, int n)
{
    NODE *new_node;
    new_node = (NODE *) malloc(sizeof(struct node));
    new_node ->data = n;
    new_node ->next = start;
    start = new_node;
}

```

```

    return start;
}
NODE* i_pos(NODE *start, int n, int position)
{
    NODE *new_node, *temp, *ptr;
    int count;
    new_node = (NODE *) malloc(sizeof(struct node));
    new_node->data = n;
    if ((start == NULL) && (position ==1))
    {
        new_node->next =NULL;
        start = new_node;
    }
    else if ((start == NULL) && (position >1))
    {
        printf("Invalid postion\n");
    }

    else
    {
        count =1;
        ptr = start;
        while((ptr != NULL) && (count !=position))
        {
            temp = ptr;
            ptr= ptr->next;
            count++;
        }
        if(ptr == NULL)
        {
            printf("Invalid postion\n");
        }
        else
        {
            new_node->next = ptr;
            temp->next = new_node;
        }
    }
    return start;
}

```

```

NODE* deletekey(NODE *start, int key)
{
    NODE *temp, *ptr;

    if (start == NULL) // No Nodes
    {
        printf("List Empty\n");
    }
    else if (start->data==key) //first node is key
    {
        printf("Item Deleted: %d\n", start->data);
        ptr=start;
        start = start->next;
        free(ptr);
    }
    else
    {
        // Traverse till key is found or till last node
        ptr=start;
        while((ptr != NULL)&&(ptr->data !=key))
        {
            temp=ptr;
            ptr = ptr->next;
        }
        if(ptr == NULL) // Searched till end of list
            printf("Invalid key\n");
        else
        {
            temp->next = ptr->next;
            printf("Item deleted: %d\n",ptr->data);
            free(ptr);
        }
    }
    return start;
}

```

```

NODE* reverse(NODE *start)
{
    NODE *prev, *temp,*ptr;
    if (start == NULL)
    {
        printf("List Empty\n");
    }
}

```

```

        return start;
    }
    if (start->next== NULL)
    {
        printf("only one Node %d\n", start->data);
        return start;
    }
    else
    {
        prev = NULL;
        ptr= start;
        while (ptr != NULL)
        {
            temp = ptr->next;
            ptr->next = prev;
            prev = ptr;
            ptr = temp;
        }
        start= prev;
        return start;
    }
}

```

```

void display(NODE *start)
{
    NODE *ptr;
    if (start == NULL)
    {
        printf("List is empty\n");
    }
    ptr = start;
    printf("The list data are \n");
    while(ptr != NULL)
    {
        printf("%d\n", ptr->data);
        ptr = ptr->next;
    }
}

NODE* searchkey(NODE *start, int key)
{
    NODE *ptr;

```

```

        if (start == NULL)
        {
            printf("List Empty\n");
        }
    else
    {
        // Traverse till key is found or till last node
        ptr = start;
        while((ptr != NULL)&&(ptr->data!=key))
        {
            ptr = ptr->next;
        }
        if(ptr == NULL) // Searched till end of list
        {
            printf("Invalid key\n");
        }
        else
        {
            printf("Key %d found \n", ptr->data);
        }
    }
    return start;
}

```

OUTPUT

Enter your choice

1. Insert at front
2. Insert at position
3. Delete a node
4. Searching a key
5. Reverse
6. Display
7. Stop

1

Enter data to insert

11

Enter your choice

1. Insert at front
2. Insert at position
3. Delete a node
4. Searching a key

5. Display

6. Stop

1

Enter data to insert

22

Enter your choice

1. Insert at front

2. Insert at position

3. Delete a node

4. Searching a key

5. Display

6. Stop

1

Enter data to insert

33

Enter your choice

1. Insert at front

2. Insert at position

3. Delete a node

4. Searching a key

5. Display

6. Stop

5

The list data are

33

22

11

Enter your choice

1. Insert at front

2. Insert at position

3. Delete a node

4. Searching a key

5. Display

6. Stop

2

Enter data to insert

44

2

Enter your choice

1. Insert at front

2. Insert at position

3. Delete a node
4. Searching a key
5. Display
6. Stop

5

The list data are

33

44

22

11

Enter your choice

1. Insert at front
2. Insert at position
3. Delete a node
4. Searching a key
5. Display
6. Stop

3

Enter the key value44

Item deleted: 44

Enter your choice

1. Insert at front
2. Insert at position
3. Delete a node
4. Searching a key
5. Display
6. Stop

5

The list data are

33

22

11

Enter your choice

1. Insert at front
2. Insert at position
3. Delete a node
4. Searching a key
5. Display
6. Stop

4

Enter key to be searched

22

Key 22 found

Enter your choice

1. Insert at front
2. Insert at position
3. Delete a node
4. Searching a key
5. Display
6. Stop

4

Enter key to be searched

55

Invalid key

Enter your choice

1. Insert at front
2. Insert at position
3. Delete a node
4. Searching a key
5. Display
6. Stop

6

5. A city transport department maintains a circular list of bus stops (Stop numbers) on a circular route. Since the route is circular, after the last stop, the bus returns to the first stop. The department wants to manage the route efficiently by adding new stops, removing discontinued stops, and displaying alternate stops for express services. Write a C program to implement a Circular Singly Linked List with the following operations:

- a. Insert a bus stop at the end of the circular route.
- b. Delete a bus stop from the front of the route.
- c. Delete a specific bus stop by its stop ID (key).
- d. Display alternate bus stops in the circular route (for express service planning).

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int data;
    struct Node *next;
};
struct Node *last = NULL;
/* Insert at the end */
void insertEnd(int value)
{
    struct Node *newNode;
    newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = value;
    if (last == NULL)
    {
        last = newNode;
        last->next = last;
    }
    else
    {
        newNode->next = last->next;
        last->next = newNode;
        last = newNode;
    }
}
/* Delete from the front */
void deleteFront()
{
    struct Node *temp;
```

```

    if (last == NULL)
    {
        printf("List is empty\n");
        return;
    }

    temp = last->next;

    if (last == temp)
    {
        last = NULL;
    }
    else
    {
        last->next = temp->next;
    }

    printf("Deleted bus stop: %d\n", temp->data);
    free(temp);
}

/* Delete a specific bus stop */
void deleteKey(int key)
{
    struct Node *current, *previous;

    if (last == NULL)
    {
        printf("List is empty\n");
        return;
    }
    current = last->next;
    previous = last;
    do
    {
        if (current->data == key)
        {
            if (current == last && current == last->next)
            {
                last = NULL;
            }
        }
    } while (current != NULL);
}

```

```

    }
    else
    {
        previous->next = current->next;
        if (current == last)
            last = previous;
    }
    free(current);
    printf("Bus stop %d deleted\n", key);
    return;
}
previous = current;
current = current->next;
} while (current != last->next);
printf("Bus stop %d not found\n", key);
}
/* Display alternate bus stops */
void displayAlternate()
{
    struct Node *temp;
    int count = 1;
    if (last == NULL)
    {
        printf("List is empty\n");
        return;
    }
    temp = last->next;
    printf("Alternate bus stops: ");
    do
    {
        if (count % 2 != 0)
            printf("%d ", temp->data);
        temp = temp->next;
        count++;
    } while (temp != last->next)
    printf("\n");
}
/* Display all stops */
void display()
{
    struct Node *temp;

```

```

if (last == NULL)
{
    printf("List is empty\n");
    return;
}
temp = last->next;
printf("Bus stops: ");
do
{
    printf("%d ", temp->data);
    temp = temp->next;
} while (temp != last->next);
printf("\n");
}
int main()
{
    int choice, value;
    while (1)
    {
        printf("\n--- Circular Singly Linked List ---\n");
        printf("1. Insert at End\n");
        printf("2. Delete from Front\n");
        printf("3. Delete by Stop ID\n");
        printf("4. Display Alternate Stops\n");
        printf("5. Display All Stops\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                printf("Enter Stop ID: ");
                scanf("%d", &value);
                insertEnd(value);
                break;
            case 2:
                deleteFront();
                break;
            case 3:
                printf("Enter Stop ID to delete: ");
                scanf("%d", &value);

```

```

        deleteKey(value);
        break;
    case 4:
        displayAlternate();
        break;
    case 5:
        display();
        break;
    case 6:
        exit(0);
    default:
        printf("Invalid choice\n");
    }
}
return 0;
}

```

Output

--- Circular Singly Linked List ---

1. Insert at End
2. Delete from Front
3. Delete by Stop ID
4. Display Alternate Stops
5. Display All Stops
6. Exit

Enter your choice: 1

Enter Stop ID: 101

Enter your choice: 1

Enter Stop ID: 102

Enter your choice: 1

Enter Stop ID: 103

Enter your choice: 1

Enter Stop ID: 104

Enter your choice: 5

Bus stops: 101 102 103 104

Enter your choice: 4

Alternate bus stops: 101 103

6. A music streaming application maintains a playlist where each song (song number) is connected to both its previous and next songs, allowing users to navigate forward and backward through the playlist. The application needs to support adding songs, inserting songs before a specific song, removing songs, and displaying the playlist.

Write a C program to implement a Doubly Linked List with the following operations:

- a. Insert a song at the front of the playlist.
- b. Insert a new song to the left (before) a specified song identified by its song ID (key).
- c. Delete a song by its song ID (value).
- d. Display all songs in the playlist from beginning to end.

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int data;
    struct Node *prev;
    struct Node *next;
};
struct Node *head = NULL;
/* Insert at front */
void insertFront(int value)
{
    struct Node *newNode;
    newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->prev = NULL;
    newNode->next = head;
    if (head != NULL)
        head->prev = newNode;
    head = newNode;
}
/* Insert before a specified song */
void insertBefore(int key, int value)
{
    struct Node *current, *newNode;
    current = head;
    while (current != NULL && current->data != key)
    {
        current = current->next;
    }
    if (current == NULL)
```

```

    {
        printf("Song %d not found\n", key);
        return;
    }
    newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = current;
    newNode->prev = current->prev;
    if (current->prev != NULL)
        current->prev->next = newNode;
    else
        head = newNode;
    current->prev = newNode;
    printf("Song inserted successfully\n");
}
/* Delete a song by ID */
void deleteSong(int key)
{
    struct Node *current;
    current = head;
    while (current != NULL && current->data != key)
    {
        current = current->next;
    }
    if (current == NULL)
    {
        printf("Song %d not found\n", key);
        return;
    }
    if (current->prev != NULL)
        current->prev->next = current->next;
    else
        head = current->next;
    if (current->next != NULL)
        current->next->prev = current->prev;
    free(current);
    printf("Song %d deleted\n", key);
}
/* Display songs */
void display()
{

```

```

struct Node *temp;
if (head == NULL)
{
    printf("Playlist is empty\n");
    return;
}
temp = head;
printf("Playlist: ");
while (temp != NULL)
{
    printf("%d ", temp->data);
    temp = temp->next;
}
printf("\n");
}
int main()
{
    int choice, value, key;
    while (1)
    {
        printf("\n--- Doubly Linked List ---\n");
        printf("1. Insert Song at Front\n");
        printf("2. Insert Song Before a Song\n");
        printf("3. Delete Song\n");
        printf("4. Display Playlist\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                printf("Enter Song ID: ");
                scanf("%d", &value);
                insertFront(value);
                break;
            case 2:
                printf("Enter Song ID before which to insert: ");
                scanf("%d", &key);
                printf("Enter new Song ID: ");
                scanf("%d", &value);
                insertBefore(key, value);

```

```

        break;
    case 3:
        printf("Enter Song ID to delete: ");
        scanf("%d", &key);
        deleteSong(key);
        break;
    case 4:
        display();
        break;
    case 5:
        exit(0);
    default:
        printf("Invalid choice\n");
    }
}
return 0;
}

```

Output:

--- Doubly Linked List ---

1. Insert Song at Front
2. Insert Song Before a Song
3. Delete Song
4. Display Playlist
5. Exit

Enter your choice: 1

Enter Song ID: 101

Enter your choice: 1

Enter Song ID: 103

Enter your choice: 2

Enter Song ID before which to insert: 103

Enter new Song ID: 102

Song inserted successfully

Enter your choice: 4

Playlist: 103 102 101

7. A travel agency maintains a list of tourist destinations for a round-trip package. Since the tour starts and ends at the same location, the destinations are organized in a circular doubly linked structure. Tour managers need to add new destinations, remove cancelled destinations, and view the itinerary in both forward and reverse directions.

Write a C program to implement a Circular Doubly Linked List with the following operations:

- a. Insert a destination at the end of the itinerary.**
- b. Delete a destination using its destination ID (key).**
- c. Display all destinations in the forward direction.**
- d. Display all destinations in the reverse direction.**

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node
{
    int data;
    struct Node *prev;
    struct Node *next;
};
```

```
struct Node *head = NULL;
```

```
/* Insert at the end */
```

```
void insertEnd(int value)
```

```
{
    struct Node *newNode;
    struct Node *last;
    newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = value;
    if (head == NULL)
    {
        head = newNode;
        newNode->next = head;
        newNode->prev = head;
    }
    else
    {
        last = head->prev;
        newNode->next = head;
        newNode->prev = last;
    }
}
```

```

        last->next = newNode;
        head->prev = newNode;
    }
}
/* Delete a destination by ID */
void deleteKey(int key)
{
    struct Node *temp;
    if (head == NULL)
    {
        printf("List is empty\n");
        return;
    }
    temp = head;
    do
    {
        if (temp->data == key)
        {
            /* Only one node */
            if (temp->next == head && temp->prev == head)
            {
                head = NULL;
            }
            else
            {
                temp->prev->next = temp->next;
                temp->next->prev = temp->prev;
                if (temp == head)
                    head = temp->next;
            }
            free(temp);
            printf("Destination %d deleted\n", key);
            return;
        }
        temp = temp->next;
    } while (temp != head);
    printf("Destination %d not found\n", key);
}
/* Display in forward direction */
void displayForward()
{

```

```

struct Node *temp;
if (head == NULL)
{
    printf("Itinerary is empty\n");
    return;
}
temp = head;
printf("Forward itinerary: ");
do
{
    printf("%d ", temp->data);
    temp = temp->next;
} while (temp != head);
printf("\n");
}
/* Display in reverse direction */
/void displayReverse()
{
    struct Node *temp;
    if (head == NULL)
    {
        printf("Itinerary is empty\n");
        return;
    }
    temp = head->prev;
    printf("Reverse itinerary: ");
    do
    {
        printf("%d ", temp->data);
        temp = temp->prev;
    } while (temp != head->prev);
    printf("\n");
}
int main()
{
    int choice, value;
    while (1)
    {
        printf("\n--- Circular Doubly Linked List ---\n");
        printf("1. Insert at End\n");
        printf("2. Delete by Destination ID\n");
    }
}

```

```

printf("3. Display Forward\n");
printf("4. Display Reverse\n");
printf("5. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice)
{
    case 1:
        printf("Enter Destination ID: ");
        scanf("%d", &value);
        insertEnd(value);
        break;
    case 2:
        printf("Enter Destination ID to delete: ");
        scanf("%d", &value);
        deleteKey(value);
        break;
    case 3:
        displayForward();
        break;
    case 4:
        displayReverse();
        break;
    case 5:
        exit(0);
    default:
        printf("Invalid choice\n");
}
}
return 0;
}

```

Output:

--- Circular Doubly Linked List ---

1. Insert at End
2. Delete by Destination ID
3. Display Forward
4. Display Reverse
5. Exit

Enter your choice: 1
Enter Destination ID: 101

Enter your choice: 1
Enter Destination ID: 102

Enter your choice: 1
Enter Destination ID: 103

Enter your choice: 3
Forward itinerary: 101 102 103

Enter your choice: 4
Reverse itinerary: 103 102 101

Enter your choice: 2
Enter Destination ID to delete: 102
Destination 102 deleted

Enter your choice: 3
Forward itinerary: 101 103

Enter your choice: 4
Reverse itinerary: 103 101

8. A web browser maintains the history of recently visited web pages. Whenever a user visits a new page, it is added to the top of the history stack. When the user presses the Back button, the most recently visited page is removed from the stack. The browser also needs to display the current history and handle situations when the history is full or empty.

Write a C program to implement a Stack using Arrays with the following operations:

- 1. Push a webpage ID onto the stack.**
- 2. Pop the most recently visited webpage.**
- 3. Display all webpages currently in the history stack.**
- 4. Handle Overflow when the stack is full.**
- 5. Handle Underflow when the stack is empty.**

```
#include<stdio.h>
#include<stdlib.h>
#define max_size 5

int stack[max_size],top=-1;

void push();
void pop();
void display();

int main()
{
    int choice,webpageID;
    while(1)
    {
        printf("\n-----Browser History-----\n");
        printf("1.Visit a new webpage (Push)\n");
        printf("2.Press Back (Pop)\n");
        printf("3.Display History\n");
        printf("4.Exit\n");
        printf("\nEnter your choice:\t");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1:
                push();
                break;
            case 2:
                pop();
                break;
```

```

        case 3:
            display();
            break;
        case 4:
            exit(0);
            break;
        default: printf("\nInvalid choice:\n");
            break;
    }
}
return 0;
}

```

void push() //Inserting element into the stack

```

{
    int id,n;
    if(top==(max_size-1))
    {
        printf("\nStack Overflow! History is full");
    }
    else
    {
        printf("Enter webpage ID:\t");
        scanf("%d",&id);
        top=top+1;
        stack[top]=id;
        printf("Webpage %d added to history\n",id);
    }
}

```

void pop() //deleting an element from the stack

```

{
    int id;
    if(top==-1)
    {
        printf("Stack Underflow! History is empty.");
    }
    else
    {
        id=stack[top];
        top=top-1;
    }
}

```

```

        printf("\Webpage %d removed from history\t",id);
    }
}

void display()
{
    int i;
    if(top== -1)
    {
        printf("\nHistory is Empty:");
    }
    else
    {
        printf("\nThe current browser history:\n" );
        for(i=top;i>=0;i--)
        {
            printf("%d\n",stack[i]);
        }
    }
}

```

Output:

-----Browser History-----

- 1.Visit a new webpage (Push)
- 2.Press Back (Pop)
- 3.Display History
- 4.Exit

Enter your choice: 1

Enter webpage ID: 123

Webpage 123 added to history

-----Browser History-----

- 1.Visit a new webpage (Push)
- 2.Press Back (Pop)
- 3.Display History
- 4.Exit

Enter your choice: 1

Enter webpage ID: 456

Webpage 456 added to history

-----Browser History-----

- 1.Visit a new webpage (Push)
- 2.Press Back (Pop)
- 3.Display History
- 4.Exit

Enter your choice: 3

The current browser history:

456
123

-----Browser History-----

- 1.Visit a new webpage (Push)
- 2.Press Back (Pop)
- 3.Display History
- 4.Exit

Enter your choice: 2

Webpage 456 removed from history

-----Browser History-----

- 1.Visit a new webpage (Push)
- 2.Press Back (Pop)
- 3.Display History
- 4.Exit

Enter your choice: 3

The current browser history:

123

-----Browser History-----

- 1.Visit a new webpage (Push)
- 2.Press Back (Pop)
- 3.Display History
- 4.Exit

Enter your choice: 4

9. A) Write a C program to convert a given infix expression into its equivalent postfix expression using a stack.

```
#include<stdio.h>
#include<ctype.h>
void infixtopostfix(char in[50]);
int precedence(char operator);
int isOperator(char ch);
int main()
{
    char infix[50];
    printf("Enter the expression:");
    scanf("%s",infix);
    infixtopostfix(infix);
    return 0;
}
int isOperator(char ch)
{
    switch (ch)
    {
        case '+':
        case '-':
        case '*':
        case '/':
        case '^':
            return 1;
        default:
            return 0;
    }
}
int precedence(char operator)
{
    switch (operator)
    {
        case '+':
        case '-':
            return 1;
        case '*':
        case '/':
            return 2;
```

```

        case '^':
            return 3;
        default:
            return -1;
    }
}

void infixtopostfix(char in[50])
{
    char stack[50], postfix[50];
    int top=-1, i, j;
    for(i=0, j=0; in[i]!='\0'; i++)
    {
        if (isalnum(in[i]))
        {
            postfix[j++] = in[i];
        }
        else if (in[i] == '(' )
        {
            stack[++top] = in[i];
        }
        else if ( in[i] == ')' )
        {
            while (top > -1 && stack[top] != '(')
            {
                postfix[j++] = stack[top--];
            }
            top--;
        }
        else if(isOperator(in[i]) )
        {
            while ((top > -1) && precedence(stack[top]) >= precedence(in[i]))
            {
                postfix[j++] = stack[top--];
            }
            stack[++top] = in[i];
        }
    }
    while (top > -1)
    {
        postfix[j++] = stack[top--];
    }
}

```

```

    }
    postfix[j] = '\0';
    printf("The corresponding postfix expression is:");
    printf("%s",postfix);
}

```

Output:

Enter the expression:a+b*c

The corresponding postfix expression is:abc*+

B) Write a C program to evaluate a given postfix expression using a stack and display the result.

```

#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#define MAX_SIZE 100
int stack[MAX_SIZE], top;
void push(int val)
{
    if (top == MAX_SIZE - 1)
    {
        printf("Error: Stack overflow.\n");
        exit(1);
    }
    top++;
    stack[top] = val;
}
int pop()
{
    if (top == -1)
    {
        printf("Error: Invalid expression \n");
        exit(1);
    }
    return stack[top--];
}
int evaluatePostfix(char expr[])
{
    top = -1;
    int result;
    for(int i=0;expr[i] != '\0';i++)

```

```

{
    if (isdigit(expr[i]))
    {
        push(expr[i]-'0');
    }
    else
    {
        int val2 = pop();
        int val1 = pop();
        switch (expr[i])
        {
            case '+': result=val1 + val2; break;
            case '-': result=val1 - val2; break;
            case '*': result=val1 * val2; break;
            case '/':
                if (val2 == 0){
                    printf("Error: Division by zero.\n");
                    exit(1);
                }
                result=val1 / val2; break;
            default:
                printf("Invlid Operator"); exit(1);
        }
        push(result);
    }
}
return pop();
}
int main()
{
    char expression[50];
    printf("Enter the postfix expression:\n");
    scanf("%s",expression);
    int result = evaluatePostfix(expression);
    printf("Evaluated Result:  %d\n", result);
    return 0;
}

```

Output:

Enter the postfix expression:

123*+

Evaluated Result: 7

10. A) Write a C program to solve the Tower of Hanoi problem using recursion and display the sequence of disk movements from the source rod to the destination rod.

```
#include <stdio.h>

void towerOfHanoi(int n, char source, char destination, char auxiliary)
{
    if (n == 1)
    {
        printf("Move disk 1 from rod %c to rod %c\n", source, destination);
        return;
    }
    towerOfHanoi(n - 1, source, auxiliary, destination);
    printf("Move disk %d from rod %c to rod %c\n", n, source, destination);
    towerOfHanoi(n - 1, auxiliary, destination, source);
}

int main()
{
    int num_disks;
    printf("Enter the number of disks: ");
    scanf("%d", &num_disks);
    if (num_disks <= 0)
    {
        printf("Please enter a positive integer for the number of disks.\n");
        return 1;
    }
    printf("\nThe sequence of moves to solve Tower of Hanoi:\n");
    towerOfHanoi(num_disks, 'A', 'C', 'B');
    return 0;
}
```

Output:

Enter the number of disks:

3

The sequence of moves to solve Tower of Hanoi:

Move disk 1 from rod A to rod C

Move disk 2 from rod A to rod B

Move disk 1 from rod C to rod B

Move disk 3 from rod A to rod C

Move disk 1 from rod B to rod A

Move disk 2 from rod B to rod C

Move disk 1 from rod A to rod C

B) A college maintains a sorted list of student roll numbers. Write a C program to search for a given roll number using Binary Search.

```
#include <stdio.h>
```

```
int binarySearchRecursive(int arr[], int low, int high, int target)
{
    if (low > high)
    {
        return -1;
    }
    int mid = low + (high - low) / 2;
    if (arr[mid] == target)
    {
        return mid;
    }
    if (arr[mid] > target)
    {
        return binarySearchRecursive(arr, low, mid - 1, target);
    }
    return binarySearchRecursive(arr, mid + 1, high, target);
}

int main()
{
    int n, target, result;
    printf("Enter the number of students: ");
    scanf("%d", &n);
    if (n <= 0)
    {
        printf("Invalid number of students.\n");
        return 1;
    }
    int roll_numbers[n];
    printf("Enter %d student roll numbers in SORTED order:\n", n);
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &roll_numbers[i]);
    }
    printf("Enter the roll number to search: ");
    scanf("%d", &target);
    result = binarySearchRecursive(roll_numbers, 0, n - 1, target);
```

```
        if (result != -1)
        {
            printf("\nRoll number %d found at position %d (index %d).\n", target, result +
1, result);
        }
        else
        {
            printf("\nRoll number %d is not present in the list.\n", target);
        }
        return 0;
    }
}
```

Output:

Enter the number of students: 5

Enter 5 student roll numbers in SORTED order:

12

13

14

15

16

Enter the roll number to search: 14

Roll number 14 found at position 3 (index 2).